

编译器设计专题实验课2025

2025.05

一、实验六：中间代码生成

赵永亮

- ▶ 实验六：中间代码生成
- ▶ 输入抽象语法树信息或其他，token，源程序等等。
- ▶ 输出中间代码表示，三元式、四元式、三地址方式均可，以下输出均可

```
input x;  
if 0 < x then  
    fact = 1  
else fact=2;  
repeat  
    fact = fact * x;  
    x = x - 1  
until x == 0;  
print fact
```

```
INPUT x  
t1 = 0  
IF t1 < x THEN I1 ELSE I2  
LABEL I1  
fact = 1  
GOTO I3  
LABEL I2  
fact = 2  
LABEL I3  
fact = fact * x  
x = x - 1  
IF x == 0 THEN I4 ELSE I3  
LABEL I4  
PRINT fact
```

例如 $X = a*b+c/d$ 的四元式序列：

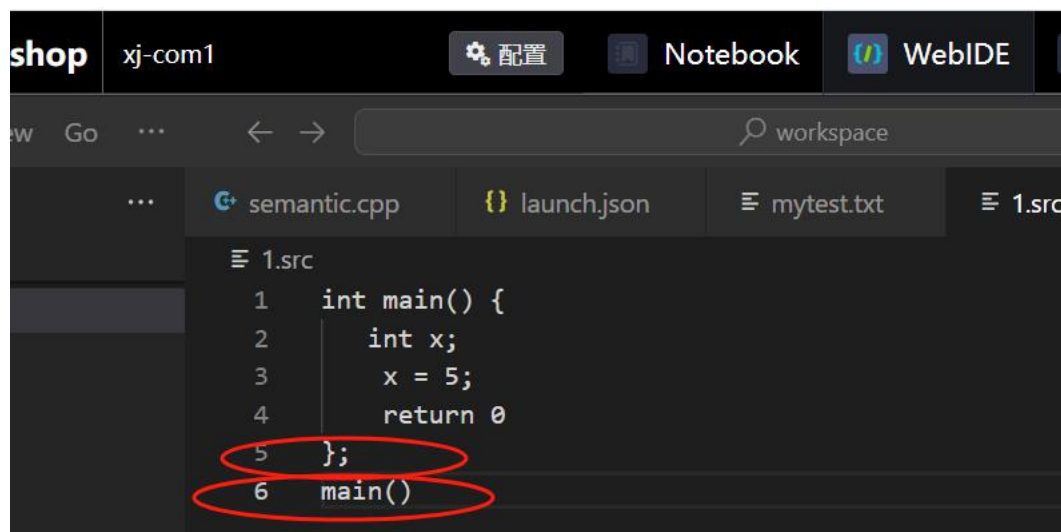
1. $(*, a, b, T1)$
2. $(/, c, d, T2)$
3. $(+, T1, T2, T3)$
4. $(=, T3, -, X)$

赵永亮

一、实验六：中间代码生成

李金亮

- ▶ 实验六：中间代码生成
- ▶ 示例代码1



```
shop xj-com1 配置 Notebook WebIDE
workspace
semantic.cpp launch.json mytest.txt 1.src
1 int main() {
2     int x;
3     x = 5;
4     return 0
5 };
6 main()
```

- ▶ 示例代码库已经上传，根据同学的修正，目前正确代码大多数，错误代码非常少，集中在词法分析阶段，能够减轻同学在出错处理阶段精力，主要用于分析文法。

李金亮

实验六：中间代码生成

2018年

▶ 示例代码库和文法的解释

▶ 程序段

1. $P \rightarrow \check{D} \check{S}$

▶ $\check{D}$ 是变量的声明语句集. $\check{S}$ 是非变量声明语句集, 包括赋值语句

▶ if 语句 while 语句, return 语句和函数调用语句

$P \rightarrow \check{D} \check{S} \leftarrow$

$\check{D} \rightarrow \varepsilon \mid \check{D} \underline{D}; \leftarrow$

$D \rightarrow T d \mid T d[i] \mid T d(\check{A}) \{ \check{D} \check{S} \} \leftarrow$

$T \rightarrow \text{int} \mid \text{float} \mid \text{void} \leftarrow$

$\check{A} \rightarrow \varepsilon \mid \check{A} \underline{A}; \leftarrow$

$A \rightarrow T d \mid T d[] \mid T d(T) \leftarrow$

$\check{S} \rightarrow S \mid \check{S}; S \leftarrow$

$S \rightarrow d = E \mid d[E] = E \mid \text{if}(\check{B}) S \mid \text{if}(B) S \text{ else } S \mid \text{while}(B) S \mid \text{return } E \mid \{ \check{S} \} \mid d(\check{R}) \mid$

$E \rightarrow i \mid f \mid d \mid d[E] \mid E + E \mid E * E \mid (E) \mid d(\check{R}) \leftarrow$

$B \rightarrow E r E \mid E \leftarrow$

$\check{R} \rightarrow \varepsilon \mid \check{R} \underline{R}; \leftarrow$

$R \rightarrow E \mid d[] \mid d() \leftarrow$

1.src - 记事本

文件(F) 编辑(E) 格式(O) 查看

int main() {

int x;

x = 5;

return 0

};

main()

函数声明

变量声明

赋值语句

return语句

关于分号的用法

S->语句列表之间用分号, 而语句结束无需分号

因此return 0 没有分号, 函数声明结束部分有分号

李银亮



二、实验步骤和结果

- ▶ 示例代码库和文法的解释
- ▶ 实验二给出的文法和大作业给出的文法有区别，但本次示例代码库无需体现细节差异，比如可以支持float，B没有如下操作
$$B \rightarrow B \wedge B \mid B \vee B \mid$$
- ▶ 如果词法支持减法 - 除法/等运算符也是允许的。因此，对于之前没有仔细分析过这个文法的细节的同学也是友好的
- ▶ 验收部分，输入是测试代码，输出可以是三元式、四元式等中间代码或者MIPS，arm等。
- ▶ 验收无需一步步显示词法、语法和语义分析结果，如果能显示属于加分项目。



二、实验步骤和结果

- ▶ 示例代码库和文法的解释（此次感谢几位同学对示例代码库的修正，5，19日更新示例代码库）
- ▶ 此外值得注意的本文法的实参和形参部分
- ▶ $A \rightarrow Td()$
- ▶ A 代表函数参数列表的声明. 可以是普通变量参数 $A \rightarrow Td$, 可以是数组参数 $A \rightarrow \tilde{A} \rightarrow \varepsilon | \tilde{A} \underline{A};$ $\tilde{R} \rightarrow \varepsilon | \tilde{R} \underline{R};$ $\tilde{A} \rightarrow Td()$
- ▶ $\tilde{A}$ 是函数参数列表的声明集. A 代表函数参数列表中某个参数的声明.
- ▶ 参数之间用分号隔开, 而不是逗号, 即使是最后一个参数也需要使用
- ▶ 分号结尾尾.

`void func1(int abc; int xyz; int arr[]; int func2();)`

- ▶ R 是函数调用参数集. R 是函数参数列表中某个参数
- ▶ 的调用. 参数之间用逗号隔开, 最后一个参数需要逗号
- ▶ 例如

```
26.src - 记事本
文件(F) 编辑(E) 格式(O) 查看(V)
int add(int a; int b;) {
    return a + b
};

int calc() {
    int sum;
    sum = add(3, 4);
    return sum
};
calc()
```



二、实验步骤和结果

➤ 示例代码库和词法的解释

$d \sim (ID, name) \leftarrow$

$ID = [a-z][a-z0-9]^* \leftarrow$

$i \sim (INT, value) \leftarrow$

$NUM = [+ -]?[0-9]^+ \leftarrow$

$f \sim (FLO, value) \leftarrow$

$FLO = [+ -]?[0-9]^* \cdot [0-9]^+ \mid [+ -]?[0-9]^+ \cdot [0-9]^* \leftarrow$

$+ \sim (ADD, -) \leftarrow$

$ADD = \{+\} \leftarrow$

$* \sim (MUL, -) \leftarrow$

$MUL = \{*\} \leftarrow$

$r \sim (ROP, name) \leftarrow$

$ROP = \{<, <=, ==\} \leftarrow$

$\backslash = \sim (ASG, -) \leftarrow$

$ASG = \{\backslash =\} \leftarrow$

$\backslash (\sim (LPA, -) \leftarrow$

$LPA = \{\backslash (\leftarrow$

$\backslash) \sim (RPA, -) \leftarrow$

$RPA = \{\backslash)\} \leftarrow$

$[\sim (LBK, -) \leftarrow$

$LBK = \{[\} \leftarrow$

$] \sim (RBK, -) \leftarrow$

$RBK = \{] \} \leftarrow$

$\backslash \{ \sim (LBR, -) \leftarrow$

$LBR = \{\backslash \{ \} \leftarrow$

$\backslash \} \sim (RBR, -) \leftarrow$

$RBR = \{\backslash \} \} \leftarrow$

$\backslash , \sim (CMA, -) \leftarrow$

$CMA = \{\backslash , \} \leftarrow$

$\backslash ; \sim (SCO, -) \leftarrow$

$SCO = \{\backslash ; \} \leftarrow$

$int \sim (INT, -) \leftarrow$

$float \sim (FLOAT, -) \leftarrow$

$if \sim (IF, -) \leftarrow$

$else \sim (ELSE, -) \leftarrow$

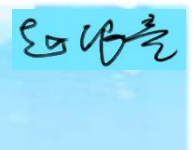
$while \sim (WHILE, -) \leftarrow$

$return \sim (RETURN, -) \leftarrow$

$input \sim (INPUT, -) \leftarrow$

$print \sim (PRINT, -) \leftarrow$

$void \sim (VOID, -) \leftarrow$



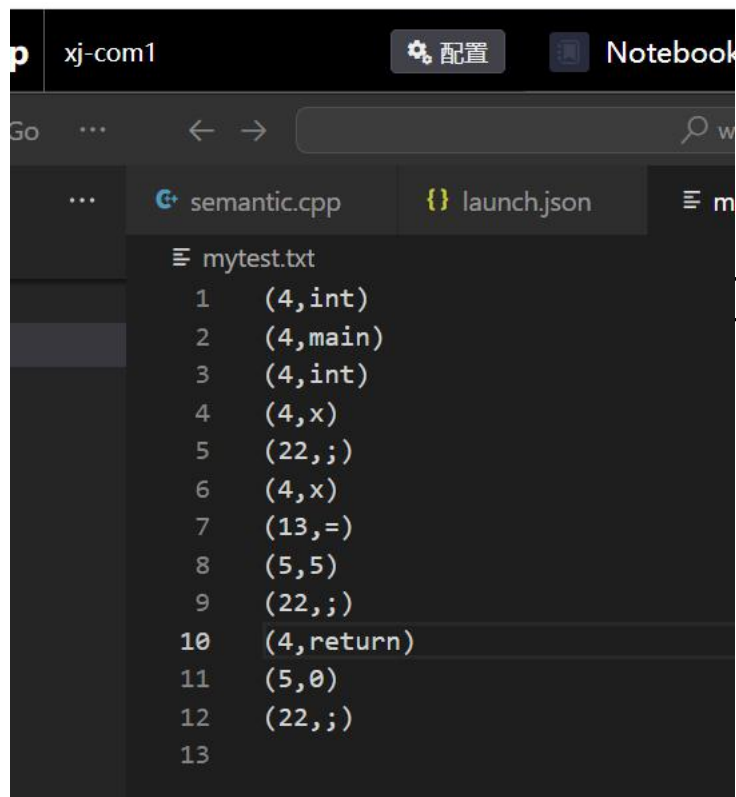


二、实验步骤和结果

▶ 示例代码库和词法的解释

- ✓ 示例程序的main被识别为ID，不是关键字
- ✓ 关键字只有int, float, void, if, else, while, return, input 和print,
- ✓ 没有 for,除了分号;和逗号，不支持双引号、单引号、\\等其他界符。
- ✓ 没有注释
- ✓ 操作符号只有<, <=, 和==，不支持大于和大于等于
- ✓ 也没有与或非等操作
- ✓ 支持，{[(,)]}
- ✓ 验收程序可以支持部分示例文法和示例词法，不需要完全一致。

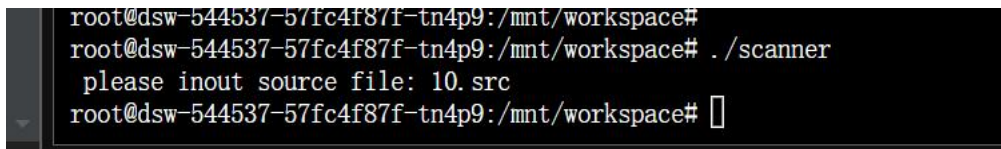
- 词法部分
- 示例程序1.src经过词法以后得到token序列



A screenshot of a code editor window titled 'xj-com1'. It shows a file named 'mytest.txt' with 13 lines of token sequences. The tokens are pairs of (line number, token string). The tokens are: (1, (4,int)), (2, (4,main)), (3, (4,int)), (4, (4,x)), (5, (22,;)), (6, (4,x)), (7, (13,=)), (8, (5,5)), (9, (22,;)), (10, (4,return)), (11, (5,0)), (12, (22,;)), and (13,).

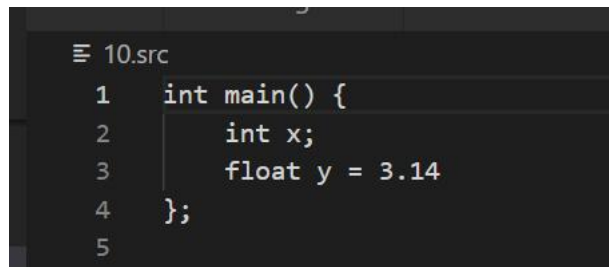
```
1 (4,int)
2 (4,main)
3 (4,int)
4 (4,x)
5 (22,;)
6 (4,x)
7 (13,=)
8 (5,5)
9 (22,;)
10 (4,return)
11 (5,0)
12 (22,;)
13
```

10.src



A screenshot of a terminal window showing the execution of a scanner program. The user runs './scanner' and the program outputs 'please inout source file: 10.src'.

```
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace#
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace# ./scanner
please inout source file: 10.src
root@dsw-544537-57fc4f87f-tn4p9:/mnt/workspace#
```



A screenshot of a code editor window showing the source code of '10.src'. The code is a C program with a main function that declares an integer 'x' and a float 'y' with the value 3.14.

```
1 int main() {
2     int x;
3     float y = 3.14
4 };
5
```



二、实验步骤和结果

- 词法部分
- 示例程序10.src经过词法以后得到token序列

```

≡ mytest.txt
1  (4,int)
2  (4,main)
3  (4,int)
4  (4,x)
5  (22,;)
6  (4,float)
7  (4,y)
8  (13,=)
9  (5,3)
10 (16,.)
11 (5,14)
12 (22,;)
13
    
```

$E \rightarrow i | f | d | d[E] | E + E | E * E | (E) | d(\check{R})$

- 表达式可以是i（整数），f（浮点数），d（某标识符）
- $E+E$ ， $E * E$ ，或者（ ），或者d（R）-函数调用表达式，或者d（E）（这个文法，大作业不支持，也可以不写）



产生式	语义规则
$S \rightarrow id = E$	$S.code = E.code$ $\parallel \text{gen}(\text{top.get}(id.lexeme) '=' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}();$ $E.code = E_1.code \parallel E_2.code$ $\parallel \text{gen}(E.addr := 'E_1.addr '+'E_2.addr)$
$E \rightarrow E_1 * E_2$	$E.addr = \text{new Temp}();$ $E.code = E_1.code \parallel E_2.code$ $\parallel \text{gen}(E.addr := 'E_1.addr '*'E_2.addr)$
$E \rightarrow (E_1)$	$E.addr = E_1.addr;$ $E.code = E_1.code$
$E \rightarrow id$	$E.addr = \text{top.get}(id.lexeme);$ $E.code = ''$



▶ 实验六输入

实验六输出

```
int main ()
{
    a=1;
    b=2;
    c=1+2*3+(4+5)*6;

    if a+b<c
    {
        a=a+b;
        b=b+a;
    }
    else
        c=1000;
}
```

```
workspace
nner-back.cpp SLR1.cpp mytestResult.txt X
mytestResult.txt
1 1. (=, 1, , a)
2 2. (=, 2, , b)
3 3. (*, 2, 3, T1)
4 4. (+, 1, T1, T2)
5 5. (+, 4, 5, T3)
6 6. (*, T3, 6, T4)
7 7. (+, T2, T4, T5)
8 8. (=, T5, , c)
9 9. (+, a, b, T6)
10 10. (j<, T6, c, 12)
11 11. (j, , , 17)
12 12. (+, a, b, T7)
13 13. (=, T7, , a)
14 14. (+, b, a, T8)
15 15. (=, T8, , b)
16 16. (j, , , 18)
17 17. (=, 1000, , c)
```

- ✓ 输入是一段符合文法的代码
- ✓ 输出是四元式方式
- ✓ 本次实验结果可作为验收结果